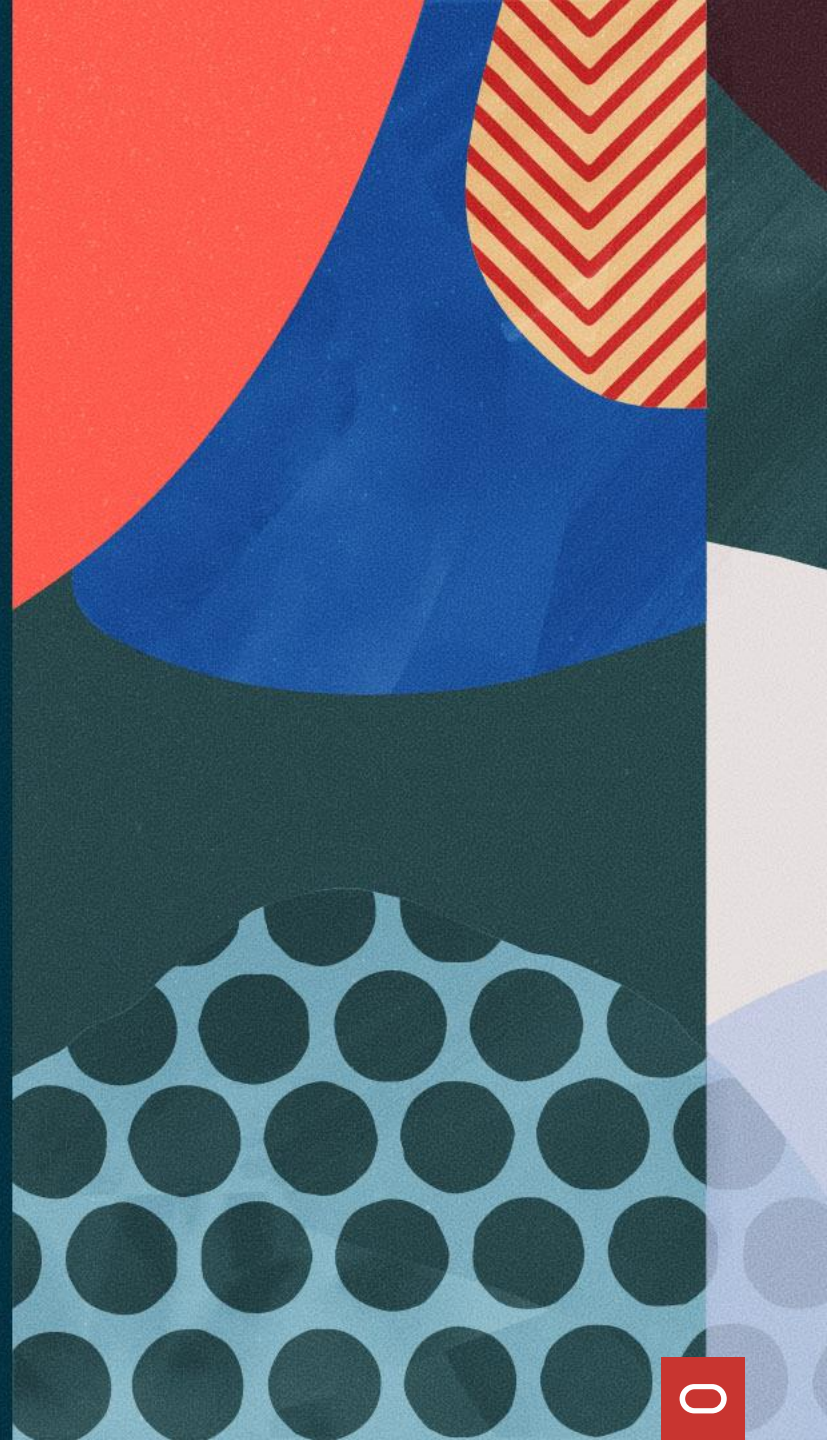
ORACLE
AI World

# SQL, JSON, and Java

SHO1351

**Josh Spiegel**

Software Architect, Oracle Database



# SQL, JSON, and Java

1 Introduction

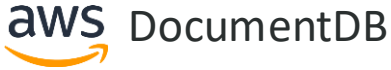
2 SQL/JSON

3 JDBC/JSON

4 Spring Data JSON

# JSON Document Databases





- Strings, numbers, Booleans arrays, and objects
- Objects/arrays can nest
- Schema independent

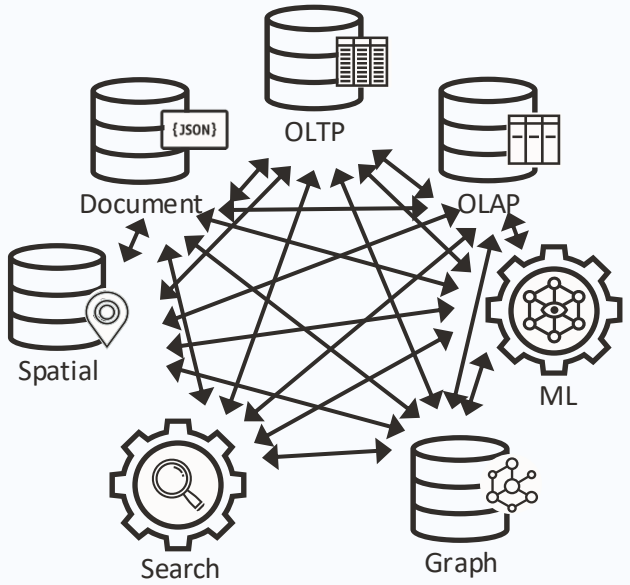
```
{
  "movie_id" : 1652,
  "title" : "Iron Man 2",
  "date" : "2010-05-07",
  "cast" : [
    "Robert Downey Jr.",
    "Larry Ellison"
  ]
}
```

# Converged Database

## Converged Database Architecture


for **any** data type or workload

## Single-purpose databases




for each data type and workload

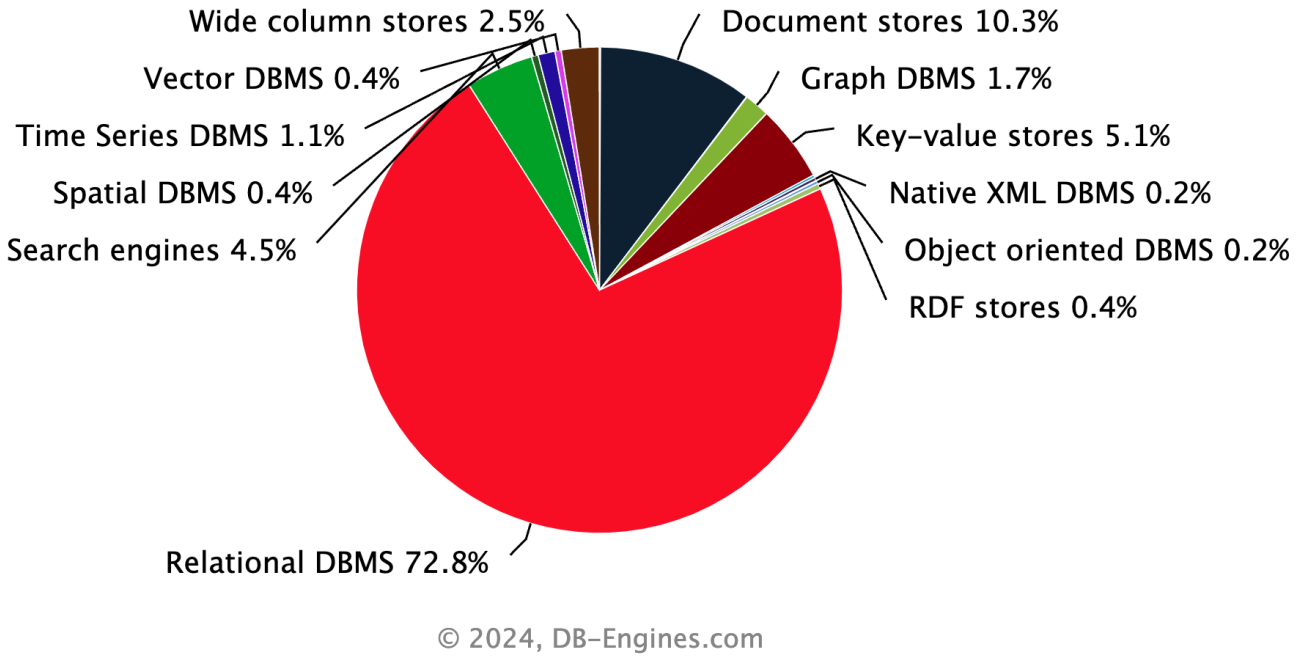
*Multiple security models, languages, skills, licenses, etc*

# Database Popularity

## Top-10 Ranked Databases

| Database | Type | Popularity |
|---|---|---|
| Oracle | Relational, Multi-model | 1244.08 |
| MySQL | Relational, Multi-model | 1061.34 |
| Microsoft SQL Server | Relational, Multi-model | 821.56 |
| PostgreSQL | Relational, Multi-model | 636.25 |
| **MongoDB** | **Document** | **421.08** |
| **Redis** | **Key-value, Multi-model** | **155.94** |
| **Elasticsearch** | **Search engine, Multi-model** | **132.83** |
| Snowflake | Relational | 130.36 |
| IBM Db2 | Relational, Multi-model | 125.90 |
| SQLite | Relational | 111.41 |

## Total Popularity % by Type



# Why store data as JSON?

```
{
  "movie_id" : 1652,
  "title" : "Iron Man 2",
  "date" : "2010-05-07",
  "cast" : [
    "Robert Downey Jr.",
    "Larry Ellison",
    ...
  ]
}
```

```
class Movie {

  int movie_id;
  String title;
  LocalTime date;
  List<String> cast;

  Movie() {
   ...
}
```

```
db.movies.insertOne(
  movieValue
);

db.movies.find(
   {"movie_id" : 1653 }
);
```

## Schema-flexible

- No upfront schema design
- Application-controlled schema
- Simple data model

## Less Impedance Mismatch

- Maps to application objects
- Supports nested structures
- Read/write without joins

## "No SQL"

- No embedding of SQL code
- Minimal new concepts for developers

# NoSQL Document Store Features

Elastic compute and storage

Single-digit latency reads and writes


Highly available


Low-cost

# Our goal: bridging the Gap between **JSON Document Store** and **Relational Worlds**

## {JSON}

Schema-flexible data
Low-latency, High throughput
Scalable, Highly available
Low-cost, developer friendly
MongoDB compatible


## SQL

SQL and analytics
In-memory columnar
Resource management
Data Normalization
ACID

# SQL/JSON

Using JSON in a relational database

# Relational Model

- A **schema** contains **tables**
- A **table** contains **rows**
- A table is **flat**
- Rows are **structured**
- Data is accessed with **SQL**
- Related rows are joined

*Flat, structured tables*

| id | title | date |
|---|---|---|
| 123 | Iron Man | 2010-05-07 |
| 345 | Thor | 2022-07-08 |

*Accessed with SQL*

```
SELECT m.title, m.date
FROM   movies m
WHERE  m.id = 123
```

# SQL/JSON

Schema-flexible JSON stored within a structured column

Defined by **ISO SQL 2016, 2023**

Stored using query-efficient native binary JSON format OSON

```
CREATE TABLE movies (data JSON);

INSERT INTO movies VALUES (
 JSON {
   'id'      : 123,
   'title'   : 'Iron Man'
 }
);

SELECT m.data.title
FROM movies m;
```

# SQL/JSON

- Use SQL to query JSON data
  - JSON to relational
  - Relational to JSON
- Joins, aggregation, projection
- Construct new JSON values
- Update JSON values
- Unnest arrays

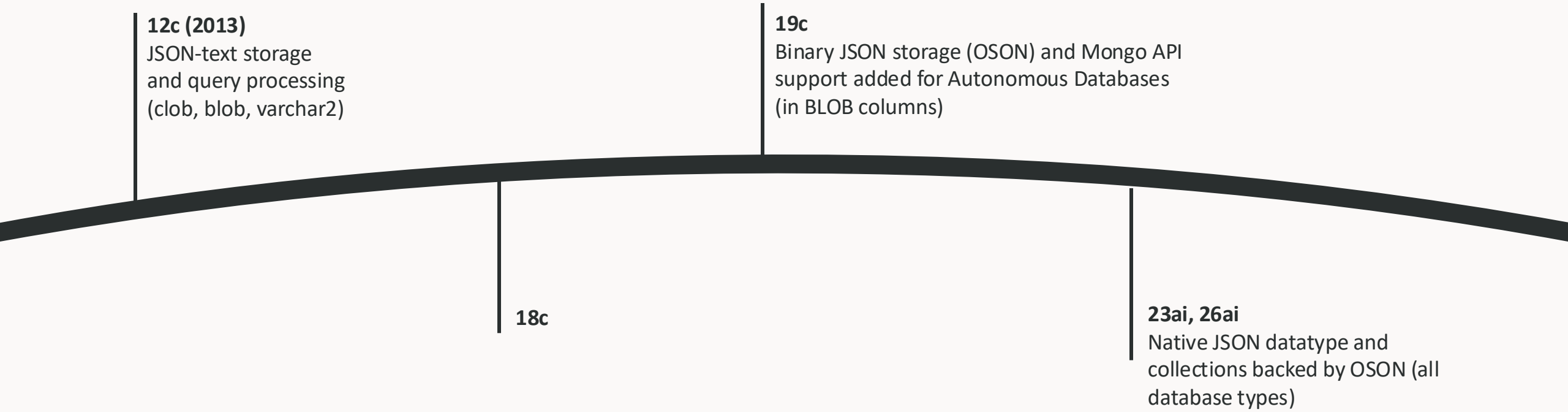
JSON aggregation and construction

```
SELECT JSON {
 'total' : sum(m.data.gross),
 'genre' : m.data.genre
}
FROM movies m
GROUP BY m.data.genre
```

JSON unnesting

```
SELECT title, actor
FROM movies NESTED data COLUMNS (
  title,
  NESTED actors[*] COLUMNS (
    actor
  )
)
WHERE id = 123
```

# Date timeline



# Why OSON?

Standard

- OBJECT { }
- ARRAY [ ]
- STRING
- TRUE/FALSE
- NULL
- NUMBER

Extended

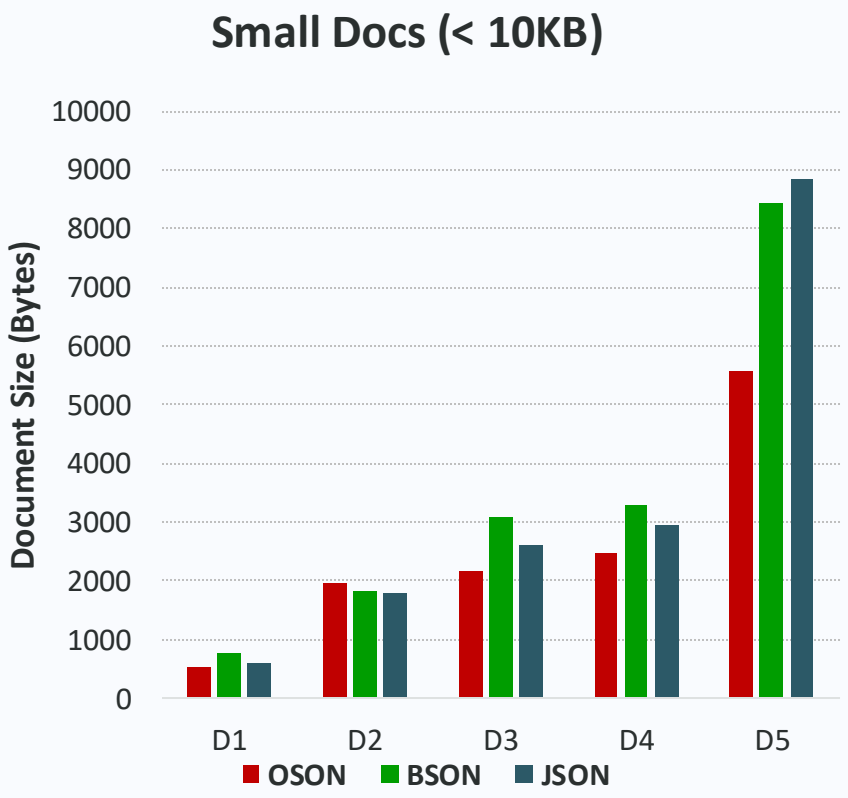
- BINARY_FLOAT
- BINARY_DOUBLE
- TIMESTAMP/DATE
- INTERVALDS/INTERVALYM
- RAW

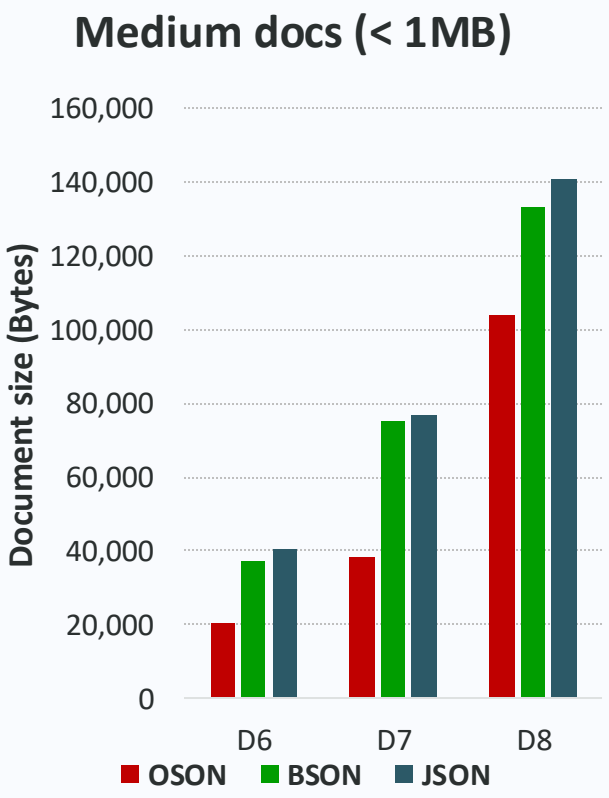
*Fidelity with relational data*

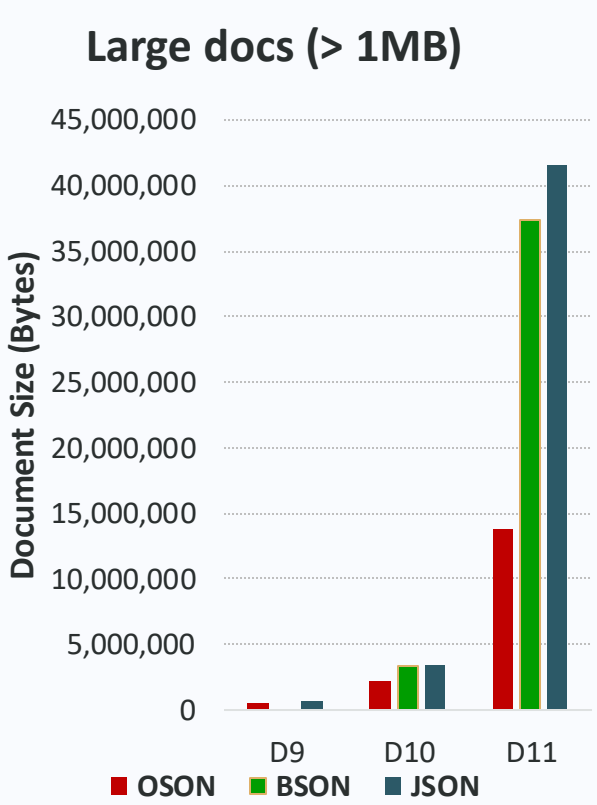
```
CREATE TABLE orders VALUES (
    oid         NUMBER,
    created     TIMESTAMP,
    status      VARCHAR2(10),
);


{
  "oid":123,
  "created":"2020-06-04T12:24:29Z",
  "status":"OPEN"
}
```

# Why OSON?

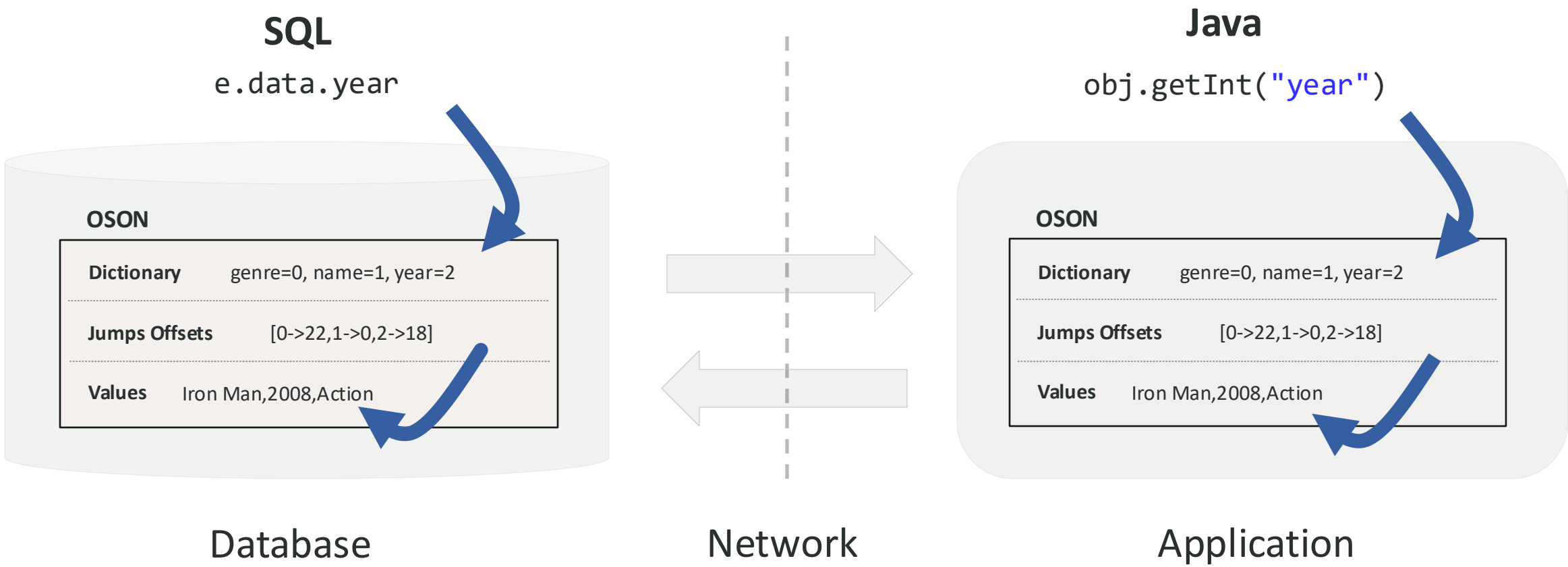


# Why OSON?

```
{"name":"Iron Man","genre":"Action","year":2008}
```

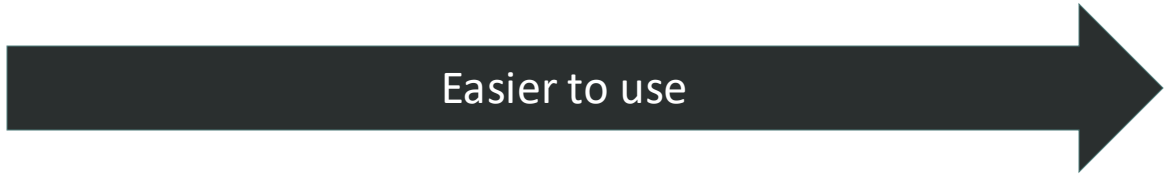


# Demo

## Creating a database and SQL/JSON

# JDBC/JSON

Accessing JSON efficiently from Java




## Streaming

Read and write JSON document as a sequence of events

Examples:

JSR-374 (jakarta.json.stream)
Jackson JsonParser/Generator

## Tree/DOM

Model JSON as a tree structure.

Examples:

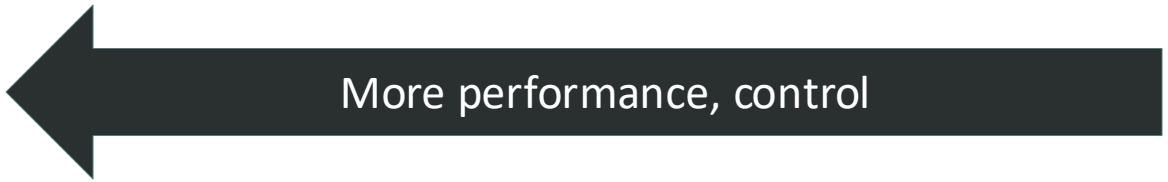
JSR-374 JSON-P (jakarta.json)
Jackson TreeNode
java.util.json (JDK preview)

## Data Binding

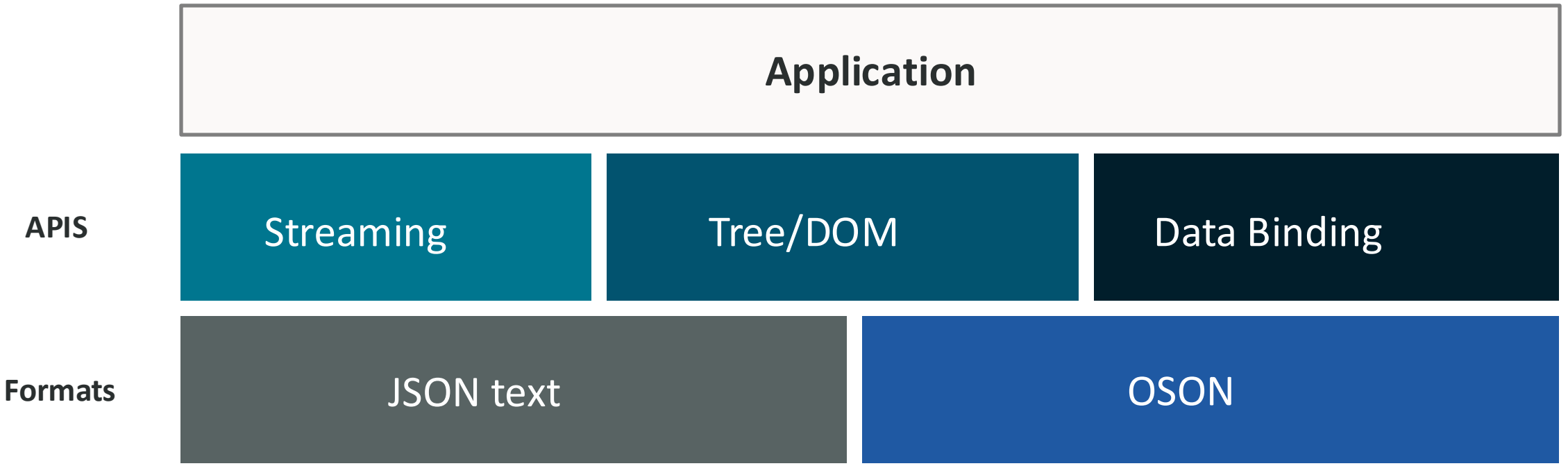
Map Java objects to and from JSON

Examples:

JSR-367 JSON-B
Jackson data-binding



# JDBC/JSON Goal: Underlying format transparent to application code



# Where's the code?

## Oracle API for JSON type

### Features

- **Streaming** and **tree** models for text and OSON
- JSON-P/JSR-374

### Location

- `Artifact: ojdbc17`
- `Group: com.oracle.database.jdbc`
- `Package: oracle.sql.json`

## Jackson Data Bind Extensions

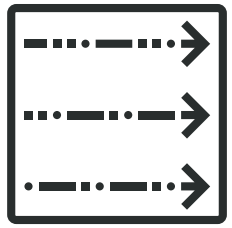
### Features

Jackson **data-bind** plugin to support direct mapping of Java objects to and from OSON

### Location

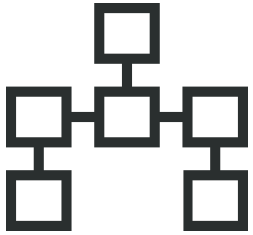
- `Artifact: ojdbc-provider-jackson-oson`
- `Group: com.oracle.database.jdbc`
- `Package: oracle.jdbc.provider.oson`

# Oracle API for JSON Type ( ojdbc17.jar : oracle.sql.json )



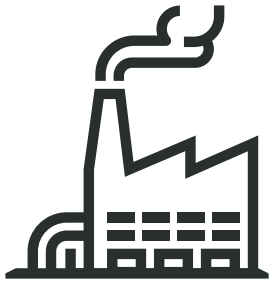
**Streaming Model**

```
OracleJsonParser
OracleJsonGenerator
```



**Tree Model**

```
OracleJsonObject
OracleJsonArray
OracleJsonString
OracleJsonDecimal
OracleJsonDouble
OracleJsonTimestamp
OracleJsonBinary
...
```



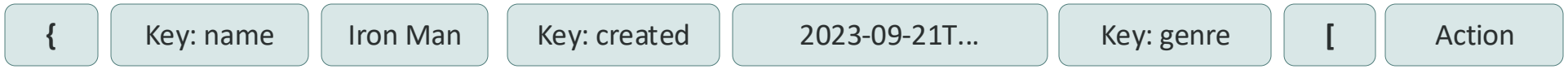
**Event Factory**

```
OracleJsonFactory
```

*optional support for JSR 374/jakarta.json*

# Streaming Model

```
{
  "name" : "Iron Man",
  "created" : "2023-09-21T21:25:05.610718",
  "genre" : ["Action", "Adventure"]
}
```




## OracleJsonParser

- Produces a sequence of events
- parser.next()
- Events can come from text or OSON

## OracleJsonGenerator

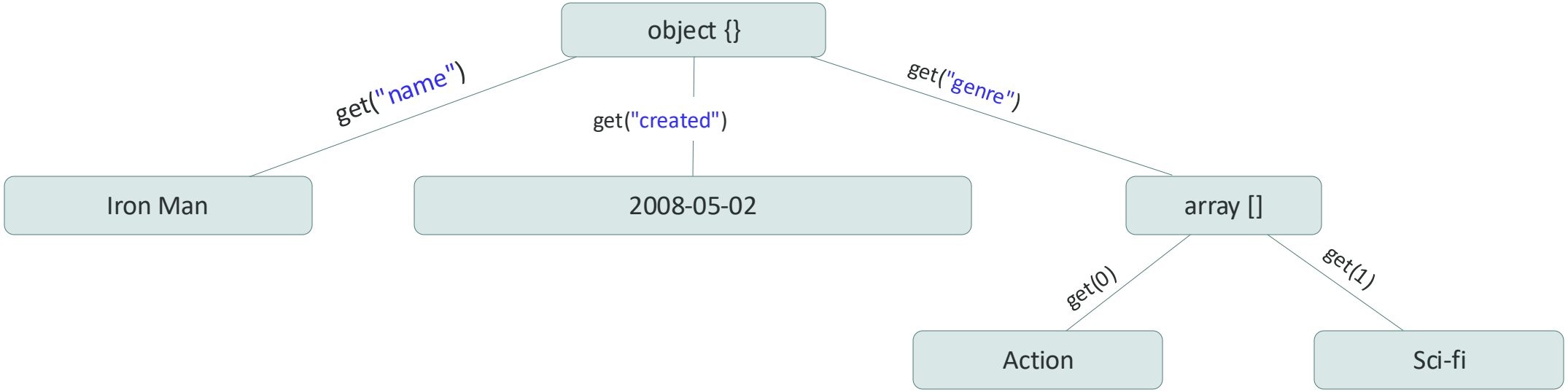
- Consumes a sequence of events
- generator.write(...)
- Generates text or OSON

# Streaming Model Example

```
OracleJsonFactory factory = new OracleJsonFactory();
FileOutputStream out = new FileOutputStream("movie.oson");
OracleJsonGenerator generator = factory.createJsonBinaryGenerator(out);

generator.writeStartObject();
generator.write("name", "Iron Man");
generator.write("genre", "Action");
generator.write("created", OffsetDateTime.now());
generator.writeEnd();

generator.close();
```

# Tree Model (or DOM – Document Object Model)

```
{
  "name" : "Iron Man",
  "created" : "2008-05-02",
  "genre" : ["Action","Sci-fi"]
}
```

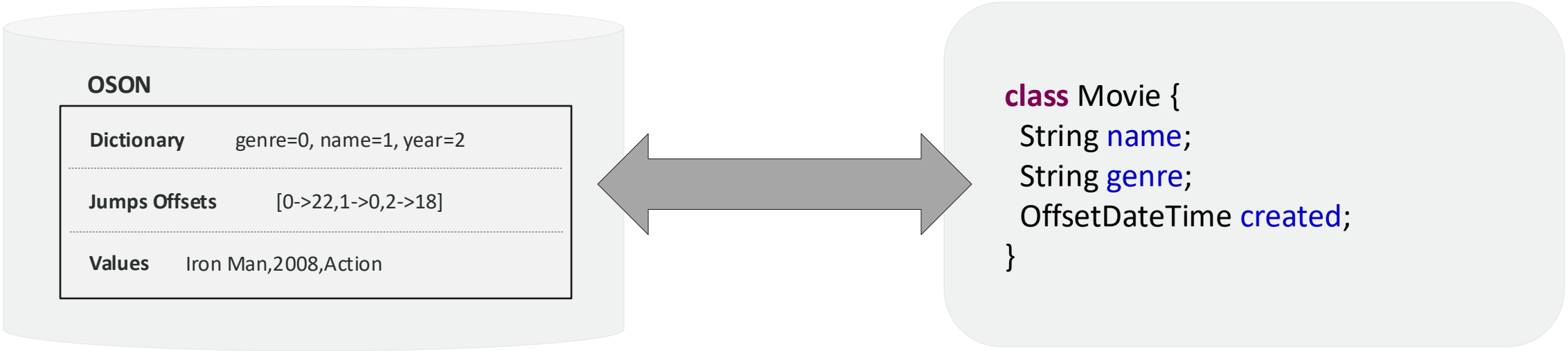


# Tree Model Example

```
OracleJsonFactory factory = new OracleJsonFactory();

OracleJsonObject movie = factory.createObject();
movie.put("name", "Iron Man");
movie.put("year", 2008);
movie.put("created", OffsetDateTime.now());

System.out.println(movie.get("name"));
System.out.println(movie.toString());
```

```
Iron Man
{"name":"Iron Man","year":2008,"created":"2023-09-21T21:25:05.610718"}
```

# Data Binding with Jackson and OSON




## Two Mapping Goals:

1. No intermediate representations
   *(e.g. no JSON text, no HashTable, etc.)*
2. Preserve types
   *(e.g. timestamps, numeric representations)*

# Jackson Data-bind Example

```
record Movie(String name, String genre, OffsetDateTime created) {}

Movie movie = new Movie("Iron Man", "Action", OffsetDateTime.now());

ObjectMapper mapper = JacksonOsonConverter.getObjectMapper();

ByteArrayOutputStream out = new ByteArrayOutputStream();
mapper.writeValue(out, movie);
byte[] oson = baos.toByteArray();

Movie movie2 = mapper.readValue(oson, Movie.class);
System.out.println(movie2.name());
System.out.println(movie2.genre());
```

```
Iron Man
Action
```

# JDBC Integration

- **SQL** statements can **consume** and and **produce** JSON values
- Relies on standard getObject() and setObject() methods throughout JDBC

# Getting JSON from the database

| SQL Methods that return JSON |
| --- |
| java.sql.ResultSet<br>.**getObject**(*, **Class<T>**)<br><br>java.sql.CallableStatement<br>.**getObject**(*, **Class<T>**) |

| Class | Content |
| --- | --- |
| java.lang.String, java.io.Reader<br>java.io.InputStream | JSON text |
| oracle.sql.json.OracleJsonValue<br>jakarta.json.JsonValue | Tree Model |
| oracle.sql.json.OracleJsonParser<br>jakarta.json.stream.JsonParser | Event Model |
| oracle.sql.json.OracleJsonDatum | Raw OSON |
| User Domain Objects<br>*(if Jackson Data Bind extensions on classpath)* | |

```
ResultSet rs = stmt.executeQuery("SELECT data FROM movies");
rs.next();
OracleJsonObject movie = rs.getObject(1, OracleJsonObject.class);
```

# Sending JSON to the database

| SQL methods accept JSON |
| --- |
| java.sql.PreparedStatement<br>.**setObject**(*, Object, SQLType)<br><br>java.sql.ResultSet<br>.**updateObject**(*, Object, SQLType)<br><br>java.sql.CallableStatement<br>.**setObject**(*, Object, SQLType)<br><br>java.sql.RowSet<br>.**setObject**(*, Object, SQLType) |

| Class | Content |
| --- | --- |
| java.lang.CharSequence, java.lang.String<br>java.io.Reader | JSON text |
| java.io.InputStream, byte[] | UTF8 or OSON |
| oracle.sql.json.OracleJsonValue<br>jakarta.json.JsonValue | Tree Model |
| oracle.sql.json.OracleJsonParser<br>jakarta.json.stream.JsonParser | Event Model |
| oracle.sql.json.OracleJsonDatum | Raw OSON |
| User Domain Objects<br>*(if DataBind extensions on classpath)* | |

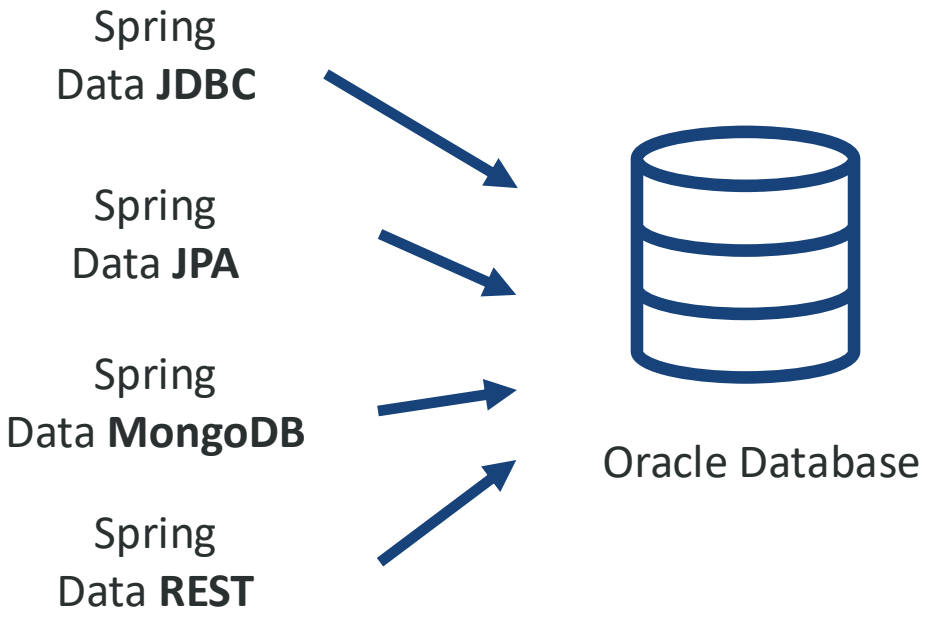
```
OracleJsonObject movie = ...;
PreparedStatement pstmt = con.prepareStatement("INSERT INTO movies VALUES (:1)");
pstmt.setObject(1, movie, OracleType.JSON);
```

# Spring Data and JSON

Using Spring Data with JSON Storage

# Spring Data

Provides a consistent programming model across different data stores.



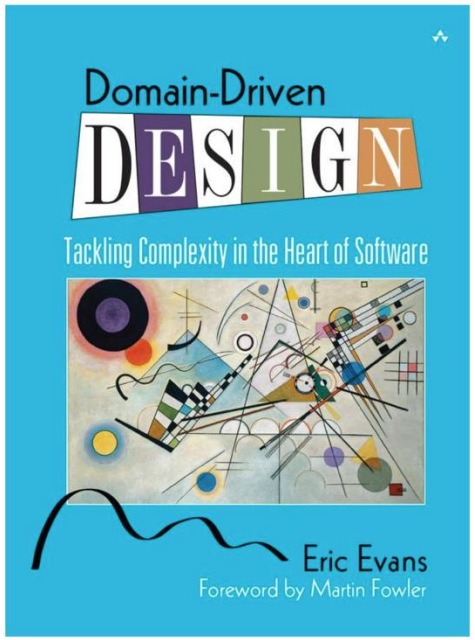

```
public class Movie {
  @Id
  Integer id;
  String name;
  String genre;
}
```

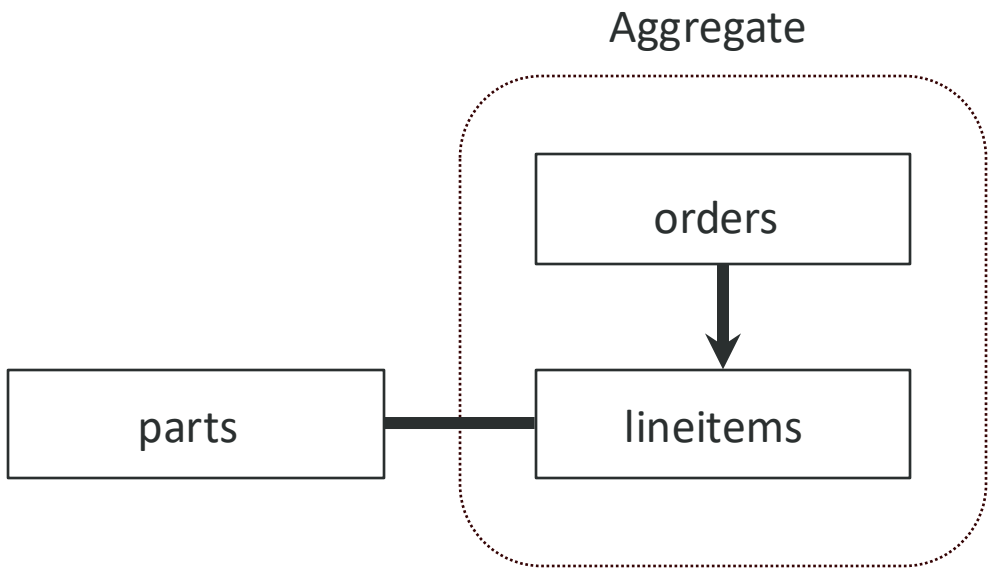
```
Movie m = movies.findById(1).get();
System.out.println(
  m.getName() + " " + m.getId()
);
```

*Oracle Database is compatible with multiple different spring data adapters.*

*Interact with your business objects as a repository of objects (**no SQL or JSON directly**)*

# Spring Data JDBC




- Spring Data JDBC implemented directly over JDBC for Oracle, MySQL, Postgres, etc.
- Inspired by domain driven design
- Repository models a collection of **aggregates**
- Aggregate root and nested entities change as a single unit of change

# Spring Data JDBC Example

```
@Data
public class Movie {
  @Id
  Integer id;
  String name;
  List<Image> images;
}
```

**MOVIES**

| ID | NAME |
|---|---|
| 1 | Iron Man |
| 2 | Interstellar |
| 3 | The Matrix |

```
@Data
public class Image {
  String file;
  String description;
}
```

**IMAGE**

| FILE | DESCRIPTION | MOVIE | MOVIE_KEY |
|---|---|---|---|
| img1.png | Robert Downey Jr. | 1 | 0 |
| img2.png | Gwyneth Paltrow | 1 | 1 |
| img3.png | Keanu Reeves | 3 | 0 |
| img4.png | Matthew McConaughey | 2 | 0 |

# Spring Data JDBC Example - findById

```
@Data
public class Movie {
 @Id
 Integer id;
 String name;
 List<Image> images;
}

@Data
public class Image {
 String file;
 String description;
}

public interface MovieRepository
  extends CrudRepository<integer, Movie>

}
```

```
Movie ironMan = movieRepo.findById(1);
```

```
SELECT
  "MOVIE"."ID" AS "ID",
  "MOVIE"."NAME" AS "NAME"
FROM "MOVIE"
WHERE "MOVIE"."ID" = :1

SELECT
  "IMAGE"."FILE" AS "FILE",
  "IMAGE"."DESCRIPTION" AS "DESCRIPTION",
  "IMAGE"."MOVIE_KEY" AS "MOVIE_KEY"
FROM "IMAGE"
WHERE "IMAGE"."MOVIE" = :1
ORDER BY "MOVIE_KEY"
```

*Default behavior, could be overridden with explicit join query.*

# Spring Data JDBC Example – save()

```
@Data
public class Movie {
  @Id
  Integer id;
  String name;
  List<Image> images;
}

@Data
public class Image {
  String file;
  String description;
}

public interface MovieRepository
   extends CrudRepository<integer, Movie>

}
```

```
movieRepo.save(newMovie);
```

```
INSERT INTO "MOVIE" ("NAME")
VALUES (:1)
RETURNING id INTO :2


INSERT INTO "IMAGE" (
  "DESCRIPTION", "FILE", "MOVIE", "MOVIE_KEY"
)
VALUES (:1, :2, :3, :4)
```

# JSON is a great fit for aggregates!

Single select, single insert

- Fewer roundtrips
- Less index maintenance
- Less random IO, block writes

| ID | NAME | IMAGES |
|---|---|---|
| 1 | Iron Man | [{"file":"img1.png"}…] |
| 2 | Interstellar | [{"file":"img3.png"}…] |
| 3 | The Matrix | [{"file":"img4.png"}…] |

Schema flexibility

- No secondary table creation
- Less "agreement" between Java and SQL required
- Attributes can change over time without modifying table definitions

Data stored in database "looks like" data in the application.

Less random IO, round-trips by default when modifying an aggregate

# Demo

# Thank you

**Josh Spiegel**

josh.spiegel@oracle.com


ORACLE